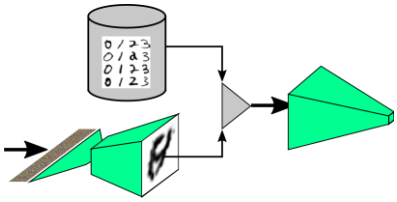
IV Construisons un GAN générateur d'images

Nous allons réaliser un petit réseau générateur qui devra générer des images de chiffres manuscrits, du même type que ceux du dataset MNIST. (Cet exercice est adapté du tutoriel suivant : https://machinelearningmastery.com/how-to-develop-a-generative-adversarial-network-for-an-mnist-handwritten-digits-from-scratch-in-keras/ -> très bon site pour aller plus loin dans le domaine du deep learning)

Notre GAN sera constitué de :

- La base d'image MNIST, qui servira de base comparative
- Un réseau convolutif générateur, qui devra générer les images
- Un générateur de vecteurs aléatoires, pour alimenter le réseau générateur
- Un réseau discriminateur, qui devra apprendre à différentier les images générées et réelles



Commençons par préparer l'environnement de travail : créez un nouveau document Colab que vous appellerez GAN1 (n'oubliez pas d'arrêter les autres sessions) et importez les librairies nécessaires :

```
import tensorflow as tf
device_name = tf.test.gpu_device_name()
if device_name != '/device:GPU:0':
  raise SystemError('GPU device not found')
print('Found GPU at: {}'.format(device_name))

from tensorflow import keras

import numpy as np
import matplotlib.pyplot as plt
```

Chargez ensuite la base MNIST. Nous utiliserons uniquement la base d'images d'entraînement :

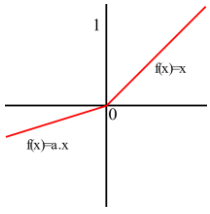
```
# on charge le dataset
(img_train, _), (_,_) = keras.datasets.mnist.load_data()
img_train = img_train / 255.0
dataset = np.expand_dims(img_train, axis=-1)

for i in range(10):
  plt.subplot(2,5,i+1)
  plt.imshow(img_train[i], cmap='gray')
plt.show()
```

On définit ensuite nos deux réseaux, en commençant par le discriminateur.

Ce réseau comportera deux couches de convolution avec 64 neurones consécutives, avec une fonction d'activation LeakyReLU, une couche Flatten et enfin une couche fully connected avec une sortie unique (un seul résultat : vrai ou fake) et une fonction d'activation sigmoïde.

La fonction d'activation LeakyReLU ne pouvant pas être utilisée en argument d'une couche de neurone (car elle nécessite ses propres paramètres), on utilisera une couche dédiée. À noter, toutes les fonctions d'activation disposent d'une couche Keras dédiée.



On compilera ensuite le réseau avec la fonction de coût BinaryCrossEntropy et la fonction d'optimisation Adam

```
discriminator_model = keras.Sequential()

discriminator_model.add(keras.layers.Conv2D(64, (3,3),input_shape=(28,28,1),
   strides=(2, 2), padding='same'))
discriminator_model.add(keras.layers.LeakyReLU(alpha=0.2))
```

```
discriminator_model.add(keras.layers.Conv2D(64, (3,3), strides=(2, 2), padding='same'))
discriminator_model.add(keras.layers.LeakyReLU(alpha=0.2))
discriminator_model.add(keras.layers.Flatten())
discriminator_model.add(keras.layers.Dense(1, activation='sigmoid'))

discriminator_model.summary()

loss=keras.losses.BinaryCrossentropy()
optim = keras.optimizers.Adam(learning_rate=0.0002, beta_1=0.5)
discriminator_model.compile(loss=loss, optimizer=optim, metrics=['accuracy'])
```

Puis on crée le réseau générateur. Pour passer d'un vecteur, dont nous définirons la taille à 100, à une image de 28x28 pixels, nous allons procéder en plusieurs étapes :

- On commence par générer une image de 7x7 pixels avec une profondeur de 128. Pour cela, on utilise une couche fully-connected avec 128x7x7 sorties, que l'on va organiser en 128 images (profondeurs) à l'aide d'une couche de type Reshape.
- On effectue ensuite deux étapes de convolution transposée/unpooling pour 'agrandir' les images jusqu'à la taille de 28x28. A noter, pour faire un unpooling de type 'bed of nails', on utilisera un stride de (2,2) dans la convolution transposée (rappel : sur la convolution transposée, le stride 'écarte' les pixels. Avec un stride de (2,2), l'image de sortie sera deux fois plus haute et large)
- Une dernière étape avec un unique neurone convolutif va générer une image de profondeur 1.

A noter : on ne compilera pas ce réseau (du moins, pas directement).

```
# espace latent à 100 dimensions
latent_dim=100

generator_model = keras.Sequential()

# on génère une image de 7x7 pixels avec une profondeur de 128
n_nodes = 128 * 7 * 7
generator_model.add(keras.layers.Dense(n_nodes, input_dim=100))
generator_model.add(keras.layers.LeakyReLU(alpha=0.2))
generator_model.add(keras.layers.Reshape((7, 7, 128)))

# convolution transposée et unpooling (7x7 -> 14x14)                    ↓ unpooling
generator_model.add(keras.layers.Conv2DTranspose(128, (4,4), strides=(2,2), padding='same'))
generator_model.add(keras.layers.LeakyReLU(alpha=0.2))

# convolution transposée et unpooling (14x14 -> 28x28)                  ↓ unpooling
generator_model.add(keras.layers.Conv2DTranspose(128, (4,4), strides=(2,2), padding='same'))
generator_model.add(keras.layers.LeakyReLU(alpha=0.2))

# convolution unique pour réduire la profondeur à 1
generator_model.add(keras.layers.Conv2D(1, (7,7), activation='sigmoid', padding='same'))

generator_model.summary()
```

On assemble ensuite le réseau générateur et le réseau discriminateur pour former le GAN. On compilera ensuite le réseau complet.

```
# connect them
gan_model = keras.Sequential()
# add generator
gan_model.add(generator_model)
# add the discriminator
gan_model.add(discriminator_model)

loss=keras.losses.BinaryCrossentropy()
optim = keras.optimizers.Adam(learning_rate=0.0002, beta_1=0.5)
gan_model.compile(loss=loss, optimizer=optim)

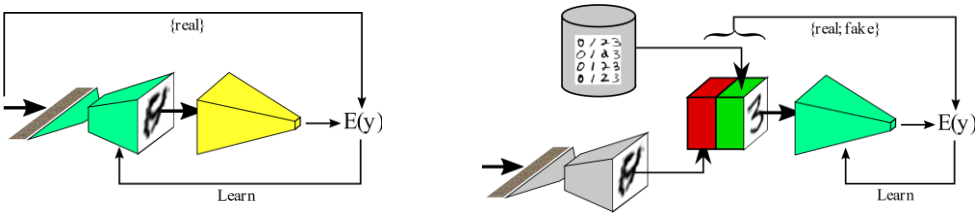
gan_model.summary()
```

Notez la structure hiérarchique des réseaux : il est toujours possible d'entraîner le discriminateur indépendamment du reste du GAN (pas le générateur puisque non compilé).

L'entraînement du GAN est particulier : il faut entraîner, de façon alternée, le réseau discriminateur sur la base d'images réelles et générées, et le réseau générateur, par le biais du GAN entier, sur la base de vecteurs aléatoires (avec les poids du discriminateur figés).

Nous allons gérer nous-même les étapes de l'apprentissage : on va se servir d'une fonction d'apprentissage de Keras : *train_on_batch*. Cette fonction permet un apprentissage sur un unique paquet d'exemples (ou *batch*), ce qui nous permettra d'entraîner de façon alterné le réseau discriminateur et le réseau générateur.

Il faut donc pouvoir générer des batchs d'apprentissage :
- Une liste de vecteurs aléatoire pour alimenter le GAN
- Une liste d'images réelles issues du dataset et des images générées par le générateur pour entrainer le réseau discriminateur



Apprentissage du générateur (à gauche) et du discriminateur (à droite)

On définit une fonction pour générer une série de vecteurs aléatoires d'une taille donnée (ici, le réseau générateur utilise des vecteurs de taille 100) :

```
# échantillon de vecteurs de l'espace latent
def generate_latent_vectors(latent_dim, n_samples):
  input_vectors = np.random.randn(latent_dim * n_samples)
  input_vectors = input_vectors.reshape(n_samples, latent_dim)
  return input_vectors
```

On définit ensuite une fonction pour récupérer un ensemble aléatoire d'images du dataset. Nous utiliserons le label '1' pour désigner une image réelle :

```
# échantillon d'images réelles
def generate_real_samples(dataset, n_samples):
  rand = np.random.randint(0, dataset.shape[0], n_samples)
  X = dataset[rand]
  # vecteur de 1 pour désigner les vraies images
  y = np.ones((n_samples, 1))
  return X, y
```

Enfin, on définit une fonction pour générer un ensemble de 'fausses' images. Pour cela, on génère un ensemble de vecteurs aléatoires qui sont envoyés dans le réseau générateur. Nous utiliserons le label '0' pour désigner une image générée :

```
# échantillon d'images générées par le réseau générateur
def generate_fake_samples(g_model, latent_dim, n_samples):
  input_vectors = generate_latent_vectors(latent_dim, n_samples)
  X = g_model.predict(input_vectors)
  y = np.zeros((n_samples, 1))
  return X, y
```

Afin de conserver une trace des images générées par notre GAN, voici une fonction pour enregistrer un échantillon de 100 images sous forme d'un fichier png. On génère un vecteur de 100 échantillons de l'espace latent qui servira à générer les images, afin de voir l'évolution, epoch après epoch, des images générées par ces mêmes 100 vecteurs.

```
# un échantillon de 100 vecteurs de l'espace latent
fake_sample_latent=generate_latent_vectors(latent_dim, 100)
```

```
# fonction pour enregistrer un échantillon d'images générées au format png
def save_image(epoch, n=10):

  # on génère les images depuis l'échantillon de vecteur
  sample_fakes=generator_model.predict(fake_sample_latent)

  # on assemble l'image
  for i in range(n * n):
    plt.subplot(n, n, 1 + i)
    plt.axis('off')
    plt.imshow(sample_fakes[i, :, :, 0], cmap='gray_r')
  # on enregistre l'image
  filename = '/tmp/generated_plot_e%03d.png' % (epoch+1)
  plt.savefig(filename)
  plt.close()
```

On effectue enfin l'apprentissage du réseau. Pour l'apprentissage du discriminateur, on construit un batch constitué à moitié d'images réelles et à moitié d'images issues du générateur. Pour l'apprentissage du générateur, on fige les poids du discriminateur, et on construit un batch de vecteurs aléatoires avec le label '1' (le résultat que l'on veut obtenir 'côté générateur') pour effectuer un entraînement sur le GAN.

La séquence suivante marche plutôt bien : deux batchs pour le discriminateur, un batch pour le générateur.

```
n_epochs=10
n_batch=256
batch_per_epoch = int(dataset.shape[0] / n_batch)
# pour chaque epoch
for i in range(n_epochs):
  # pour chaque batch
  for j in range(batch_per_epoch):

    # étape 1 : on entraine le réseau discriminateur

    # on créé un batch d'échantillon à partir d'images réelles et générées
    X_real, y_real = generate_real_samples(dataset, int(n_batch/2))
    X_fake, y_fake = generate_fake_samples(generator_model, latent_dim, int(n_batch/2))
    X, y = np.vstack((X_real, X_fake)), np.vstack((y_real, y_fake))

    # on entraine le réseau discriminateur
    discriminator_model.trainable=True
    discriminator_model.train_on_batch(X, y)

    # second batch pour entrainer le discriminateur
    X_real, y_real = generate_real_samples(dataset, int(n_batch/2))
    X_fake, y_fake = generate_fake_samples(generator_model, latent_dim, int(n_batch/2))
    X, y = np.vstack((X_real, X_fake)), np.vstack((y_real, y_fake))
    d_loss, _ = discriminator_model.train_on_batch(X, y)

    # étape 2 : on entraine le GAN entier (discriminateur figé)

    # on créé un batch d'échantillons à partir d'images générées
    X_gan = generate_latent_vectors(latent_dim, n_batch)
    y_gan = np.ones((n_batch, 1))

    # on fige le discriminateur et on effectue l'apprentissage
    discriminator_model.trainable=False
    g_loss = gan_model.train_on_batch(X_gan, y_gan)

    print('>%d, %d/%d, d=%.3f, g=%.3f' % (i+1, j+1, batch_per_epoch, d_loss, g_loss))

  # à chaque fin d'epoch, on enregistre une image d'échantillons
  save_image(i)
```

À la fin de chaque batch, vous pourrez observer les images générées dans le répertoire /tmp. Vous pourrez ensuite augmenter le nombre d'epoch (comptez environ 1min par epoch).

N'oubliez pas d'arrêter vos sessions Colab après usage !